

# DSBDA Lab - Complete Q&A Study Guide

Data Science and Big Data Analytics

Assignments 1 to 10 | Basic to Advanced | Easy Language

---

Covers: Python Libraries | Data Preprocessing | Statistics | Linear Regression | Logistic Regression  
| Naive Bayes | Text Analytics | Data Visualization

# ASSIGNMENT 1 – Data Preprocessing with Pandas (Iris Dataset)

## Basic Questions

### Q: What is a Dataset? What are its key components?

A **dataset** is a collection of records, similar to a table in a database. Key components:

- **Instance:** A single row of data (one observation).
- **Feature/Attribute:** A single column of data. It describes one property of an observation.
- **Training Dataset:** Data used to train a machine learning model.
- **Testing Dataset:** Data used to check how accurate the trained model is.

### Q: What is a DataFrame in Pandas? Explain with a simple example.

A **DataFrame** is a 2D table-like data structure in Pandas with rows and columns, like an Excel sheet.

Example: If you load the Iris dataset, you get a DataFrame with columns like Sepal\_Length, Sepal\_Width, Species, etc., and each row is one flower's measurement.

Common DataFrame functions:

- **df.head()** – shows first 5 rows
- **df.shape** – shows (rows, columns)
- **df.describe()** – shows statistics like mean, std, min, max
- **df.dtypes** – shows data type of each column

### Q: Name the 5 Python libraries used in Data Science and their main uses.

1. **Pandas:** Loading, cleaning, and manipulating data tables (DataFrames).
2. **NumPy:** Numerical computations, array operations, math functions.
3. **Matplotlib:** Plotting graphs – bar charts, line graphs, histograms, scatter plots.
4. **Seaborn:** Advanced statistical visualizations built on top of Matplotlib.
5. **Scikit-Learn:** Machine learning – classification, regression, clustering, preprocessing.

### Q: What is the Iris Dataset?

The **Iris dataset** is a famous dataset with 150 rows and 6 columns:

- Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species
- It has 3 species: Iris-setosa, Iris-versicolor, Iris-virginica (50 samples each).

It is used for classification tasks and is downloaded from the UCI Machine Learning Repository.

## Intermediate Questions

### Q: What are missing values? How do you check them in Pandas?

Missing values are empty/absent values in a dataset, represented as **NaN** (Not a Number) or **None** in Python.

Methods to check:

- **df.isnull()** – returns True where value is missing
- **df.isnull().sum()** – count of missing values per column
- **df.isnull().sum().sum()** – total missing values in entire dataset
- **df.isnull().any()** – True for columns that have at least one missing value

### Q: What are the different techniques for handling missing data?

1. **Drop rows/columns:** Use `df.dropna()` to remove rows with missing values.
2. **Fill with a constant:** `df.fillna(0)` replaces all NaN with 0.
3. **Fill with mean:** `df[col].fillna(df[col].mean())` – good for numerical data.

**4. Fill with median:** `df['col'].fillna(df['col'].median())` – better when outliers exist.

**5. Replace:** `df.replace(np.nan, -99)` – replace NaN with a specific value.

Median filling is preferred when outliers are present because mean gets affected by outliers.

### Q: What is Data Normalization? Why is it needed?

**Data normalization** (also called Min-Max scaling) converts all numerical values to a common scale, usually between 0 and 1.

**Why needed?** When features have very different ranges (e.g., age: 0-100, salary: 10000-100000), machine learning algorithms give more importance to larger values. Normalization removes this bias.

Formula:  $x\_scaled = (x - x\_min) / (x\_max - x\_min)$

In Python: Use **`preprocessing.MinMaxScaler()`** from sklearn.

### Q: What is Label Encoding? What is its limitation?

**Label Encoding** converts text/categorical values to numbers.

Example: Iris-setosa=0, Iris-versicolor=1, Iris-virginica=2

Code: **`label_encoder.fit_transform(df['Species'])`**

**Limitation:** It assigns numbers (0,1,2) which creates a false 'order' or 'priority'. The algorithm may think category 2 is 'greater' than category 0, which is wrong. Use One-Hot Encoding to avoid this.

### Q: What is One-Hot Encoding? How is it different from Label Encoding?

**One-Hot Encoding** creates new binary (0/1) columns for each category.

Example: For 3 species, 3 new columns are created:

- Iris-setosa: [1, 0, 0]
- Iris-versicolor: [0, 1, 0]
- Iris-virginica: [0, 0, 1]

**Difference from Label Encoding:** No false order is created. Each category is treated equally. Use **`OneHotEncoder()`** from sklearn or **`pd.get_dummies()`**.

## Advanced Questions

### Q: What is Dummy Variable Encoding? How is it different from One-Hot Encoding?

**Dummy Encoding** creates k-1 columns instead of k columns (where k = number of categories).

Example: For 3 colors (Red, Green, Blue), only 2 columns are needed:

- Red=[1,0], Green=[0,1], Blue=[0,0]

The third category is implied when all dummy columns are 0. This avoids the 'dummy variable trap' (multicollinearity). Use **`pd.get_dummies(df, drop_first=True)`**.

### Q: What is Data Formatting? How do you change a column's data type?

**Data Formatting** ensures all columns have the correct data type (integer, float, string, date, etc.).

To check: **`df.dtypes`**

To change: **`df['column'] = df['column'].astype('int')`**

This is important because if a numeric column is stored as 'object', mathematical operations will fail.

### Q: What are the main Pandas data types?

- **object:** Text/string or mixed types
- **int64:** Integer numbers
- **float64:** Decimal numbers
- **bool:** True/False values
- **datetime64:** Date and time values

- **category:** Finite list of repeated text values (memory-efficient)

## ASSIGNMENT 2 – Null Values, Outliers & Data Transformation

### Basic Questions

#### Q: What is an Outlier? Why do outliers occur?

An **outlier** is a data point that is very different from the rest of the data – unusually high or low.

Example: In the set [15, 101, 18, 7, 13, 16, 11], 101 is an outlier.

##### Causes:

- Human/data entry errors
- Measurement errors
- Natural variability in data
- Experimental errors

**Effect:** Outliers mainly affect the **Mean** and Standard Deviation. Median is not affected much.

#### Q: How is a Dataset created using Microsoft Excel for DSBDA?

Steps:

1. Open Excel → Save As → choose **CSV (MS-DOS)** format
2. Enter column names (features) as headers
3. Use **RANDBETWEEN(min, max)** function to fill random values
4. Add 20% 'impurities' (wrong values) intentionally for practice
5. Use nested IF formula for the response variable (e.g., placement offer count based on score)

The sample dataset used: **StudentsPerformance** with features Math\_Score, Reading\_Score, Writing\_Score, Placement\_Score, Club\_Join\_Date.

### Intermediate Questions

#### Q: Explain 4 methods to detect outliers.

1. **Boxplot:** Visual method. Points beyond the whiskers ( $1.5 \times \text{IQR}$  above Q3 or below Q1) are outliers. Use `df.boxplot()`.
2. **Scatter Plot:** Points that fall far from the main cluster are outliers. Good for seeing relationship between two variables.
3. **Z-Score:** Measures how many standard deviations a value is from the mean. Formula:  $Z = (x - \text{mean}) / \text{std}$ . Values with  $|Z| > \text{threshold}$  (e.g., 3) are outliers.
4. **IQR Method:** Most reliable method.
  - Q1 = 25th percentile, Q3 = 75th percentile
  - $\text{IQR} = Q3 - Q1$
  - Lower bound =  $Q1 - 1.5 \times \text{IQR}$
  - Upper bound =  $Q3 + 1.5 \times \text{IQR}$
  - Values outside these bounds are outliers.

#### Q: Explain 3 methods to handle (treat) outliers.

1. **Trimming/Removal:** Simply delete the outlier rows from the dataset using `df.drop(index)`. Not recommended if data is small.
2. **Quantile-Based Capping:** Replace values above the 90th percentile with the 90th percentile value (cap), and below 10th percentile with the 10th percentile value (floor). This keeps the data but limits extreme values.
3. **Mean/Median Imputation:** Replace outlier values with the median of that column. Since outliers affect the mean, we use median. Code: `df['col'] = np.where(df['col'] > upr_bound, median, df['col'])`

### Q: Write the algorithm to find the IQR and detect outliers.

Step 1: Sort the data

Step 2: Calculate  $Q1 = np.percentile(data, 25)$

Step 3: Calculate  $Q3 = np.percentile(data, 75)$

Step 4:  $IQR = Q3 - Q1$

Step 5: Lower bound =  $Q1 - 1.5 \times IQR$

Step 6: Upper bound =  $Q3 + 1.5 \times IQR$

Step 7: Any value below lower bound or above upper bound is an outlier.

## Advanced Questions

### Q: What is Data Transformation? Explain its types.

**Data Transformation** converts raw data into a better format for analysis and model building.

**Types:**

- **Smoothing:** Removes noise from data. Highlights important patterns.
- **Aggregation:** Summarizes data (e.g., daily sales summed to monthly sales).
- **Generalization:** Converts low-level data to high-level (e.g., age 22,25 → 'Young').
- **Normalization:** Scales values to 0-1 range. Types: Min-Max, Z-score, Decimal Scaling.
- **Attribute Construction:** Creates new features from existing ones (e.g., Club\_Join\_Date → Duration in years).

### Q: What is data skewness? How do you reduce it?

**Skewness** is when data is not symmetrically distributed (the bell curve is lopsided).

- **Positive Skew:** Tail on the right; mean > median. Most data on left side.
- **Negative Skew:** Tail on the left; mean < median. Most data on right side.
- **Normal Distribution:** Symmetric bell curve, mean = median = mode.

**How to reduce skewness:** Apply **log transformation**:

```
df['log_math'] = np.log10(df['math score'])
```

Log transformation compresses large values and stretches small values, making the distribution more normal (symmetric).

### Q: Write an algorithm to replace outlier values with the mean of the variable.

Step 1: Calculate mean of the column: `mean_val = df['col'].mean()`

Step 2: Find Z-scores: `z = np.abs(stats.zscore(df['col']))`

Step 3: Set threshold (e.g., 2)

Step 4: Replace outliers: `df['col'] = np.where(z > threshold, mean_val, df['col'])`

Step 5: Verify by checking the boxplot again – the extreme points should be gone.

## ASSIGNMENT 3 – Summary Statistics & Descriptive Analysis

### Basic Questions

#### Q: What is Statistics? What are its two main branches?

**Statistics** is the science of collecting, analyzing, and interpreting data to make predictions about a population from a sample.

1. **Descriptive Statistics:** Summarizes the data using numbers like mean, median, mode. It only describes the available data – no predictions.
2. **Inferential Statistics:** Uses a sample to make inferences and predictions about the entire population.

#### Q: What are Measures of Central Tendency? Explain Mean, Median, Mode.

These are single number summaries that describe the 'center' of data.

**Mean:** Sum of all values ÷ total number of values. Affected by outliers.

Example: [17,16,21,18,15,17,21,19,11,23] → Mean =  $178/10 = 17.8$

**Median:** Middle value when data is sorted. Not affected by outliers.

For even count: average of 2 middle values. For [11,15,16,17,17,18,19,21,21,23] → Median =  $(17+18)/2 = 17.5$

**Mode:** Most frequently occurring value. Can have multiple modes.

In above data: 17 and 21 both appear twice → Bimodal

#### Q: When is Mean, Median, and Mode used?

- **Mean:** When data has no outliers – gives the most accurate average.
- **Median:** When data has outliers – median is not affected by extreme values.
- **Mode:** When data has outliers AND more than half the data is the same value.

### Intermediate Questions

#### Q: What are Measures of Dispersion? Explain Variance, Standard Deviation, and Range.

These describe how 'spread out' the data is around the center.

**Range:** Maximum value - Minimum value. Simple but sensitive to outliers.

**Mean Absolute Deviation (MAD):** Average of absolute differences from mean.

**Variance:** Average of squared differences from mean. Tells how spread the data is.

**Standard Deviation:** Square root of Variance. Same unit as original data. More interpretable.

High std dev = data is widely spread. Low std dev = data is close to mean.

Python: `df.std()`, `df.var()`, `df.max() - df.min()`

#### Q: What are Quartiles? Explain Q1, Q2, Q3.

Quartiles divide data into 4 equal parts after sorting:

- **Q1 (25th percentile):** 25% of data is below this value.
- **Q2 (50th percentile):** 50% below – this is the Median.
- **Q3 (75th percentile):** 75% of data is below this value.

Python: `np.percentile(data, 25)`, `np.percentile(data, 75)`

Quartiles are used in box plots to visualize data distribution.

#### Q: What are the different types of variables?

**Categorical (Qualitative) Variables:**

- **Nominal:** Categories with no natural order. Example: Gender (Male/Female), Colors.
- **Ordinal:** Categories with a natural order. Example: Ratings (Good < Very Good < Excellent).

### Numerical (Quantitative) Variables:

- **Continuous:** Can take infinite values in a range. Example: Height (1.6321... metres), Temperature.
- **Discrete:** Finite number of values. Example: Number of students (only whole numbers), Score on a 0-10 scale.

## Advanced Questions

### Q: What is Skewness? Explain positive and negative skew.

**Skewness** measures the asymmetry of a probability distribution.

- **Zero Skewness:** Symmetric distribution (normal bell curve). Mean = Median = Mode.
- **Positive Skew (Right Skew):** Tail on the right side is longer. Mean > Mode. Most values are small, a few are very large.
- **Negative Skew (Left Skew):** Tail on the left is longer. Mean < Mode. Most values are large, a few are very small.

Python: `df.skew()` gives the skewness value.

### Q: Write Python code to calculate and display summary statistics grouped by a category.

To find mean income grouped by gender:

```
df.groupby(['Genre'])['Income'].mean()
```

To display all statistics for a specific species in Iris dataset:

```
irisSet = (iris['Species'] == 'Iris-setosa')
```

```
iris[irisSet].describe()
```

The `describe()` function shows: count, mean, std, min, 25%, 50%, 75%, max for each numeric column.

### Q: Which Pandas method is used to display statistical summary of a DataFrame?

**df.describe()** – Shows count, mean, std, min, 25%, 50%, 75%, max for all numeric columns.

**df.describe(include='all')** – Also includes statistics for non-numeric (object/categorical) columns like count, unique, top (most frequent), freq.

It is the quickest way to get an overview of your dataset's distribution.



## ASSIGNMENT 4 – Linear Regression

### Basic Questions

#### Q: What is Linear Regression?

**Linear Regression** is a supervised machine learning algorithm that finds a straight-line relationship between input (independent) variables and an output (dependent) variable.

Equation:  $Y = m \cdot X + b + e$

Where: Y = predicted output, X = input, m = slope, b = intercept (y-value when X=0), e = error.

It is used to predict continuous values like house prices, stock prices, exam scores.

#### Q: What is the difference between Univariate and Multivariate Linear Regression?

**Univariate (Simple) Linear Regression:** Uses only ONE independent variable to predict the output.

Example: Predicting XII score using X std score.

Equation:  $Y = a + bX$

**Multivariate Linear Regression:** Uses TWO or MORE independent variables.

Example: Predicting house price using area, location, and rooms.

Equation:  $Y = a + bX + cX^2$  (Polynomial form is used for better fitting)

#### Q: What is the Least Squares Method?

The **Least Squares Method** finds the best-fit line by minimizing the sum of squared errors (differences between actual and predicted values).

Formulas:

$\beta_1$  (slope) =  $\frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$

$\beta_0$  (intercept) =  $\bar{y} - \beta_1 \times \bar{x}$

Where  $\bar{x}$  = mean of X,  $\bar{y}$  = mean of Y

### Intermediate Questions

#### Q: Explain MSE, RMSE, and R-Squared as performance metrics for Linear Regression.

**MSE (Mean Squared Error):** Average of squared differences between actual and predicted values. Lower MSE = better model. MSE = 0 means perfect prediction.

Formula:  $MSE = (1/n) \times \sum(\text{actual} - \text{predicted})^2$

**RMSE (Root Mean Squared Error):** Square root of MSE. Same unit as the target variable, so easier to interpret.

Formula:  $RMSE = \sqrt{MSE}$

**R-Squared ( $R^2$ ):** Shows how much variance in Y is explained by the model. Range: 0 to 1.

$R^2$  closer to 1 = model explains most of the variation = good model.

$R^2$  closer to 0 = model is poor.

$R^2 = SSR/SST = 1 - SSE/SST$

#### Q: What is SST, SSR, and SSE?

- **SST (Total Sum of Squares):** Total variation in actual Y values from the mean.

$SST = \sum(y_i - \bar{y})^2$

- **SSR (Regression Sum of Squares):** Variation explained BY the model.

$SSR = \sum(\hat{y}_i - \bar{y})^2$

- **SSE (Sum of Squared Errors):** Variation NOT explained by the model (residual error).

$$SSE = \sum (y_i - \hat{y}_i)^2$$

Relationship:  $SST = SSR + SSE$

### Q: What is Training data and Testing data? What is Generalization?

**Training Data:** Data fed into the ML algorithm to build/train the model. The model learns patterns from this data.

**Testing Data:** Separate data used to evaluate how well the trained model performs on unseen data.

Training and testing datasets must be completely different.

**Generalization:** The model's ability to predict correctly on new, unseen data (not just the training data). A good model has low generalization error.

Usually 80% data → training, 20% data → testing. Use `train_test_split()` from sklearn.

### Advanced Questions

#### Q: Compute the Linear Regression equation for these 5 students (example).

Given:  $x = 78$ ,  $y = 77$ ,  $\sum (x_i - \bar{x})(y_i - \bar{y}) = 470$ ,  $\sum (x_i - \bar{x})^2 = 730$

$$\beta_1 = 470/730 = 0.644$$

$$\beta_0 = 77 - (0.644 \times 78) = 26.768$$

Regression equation:  $\hat{y} = 26.76 + 0.644X$

Interpretation: For every 1 unit increase in X score, Y score increases by 0.644 units.

If  $X = 80$ :  $\hat{y} = 26.76 + 0.644 \times 80 = 78.28$

#### Q: Write Python code for Linear Regression using sklearn on the Boston dataset.

Key steps:

1. Load dataset: `from sklearn.datasets import load_boston`
2. Split: `train_test_split(X, y, test_size=0.2)`
3. Train: `lm = LinearRegression(); lm.fit(X_train, y_train)`
4. Predict: `y_pred = lm.predict(X_test)`
5. Evaluate: `r2_score(y_test, y_pred), mean_squared_error(y_test, y_pred)`

## ASSIGNMENT 5 – Logistic Regression & Confusion Matrix

## Basic Questions

## Q: What is Logistic Regression? How is it different from Linear Regression?

**Logistic Regression** is a supervised ML algorithm used for **classification** (predicting categories), not continuous values.

It uses a **Sigmoid function** to output probabilities between 0 and 1.

If probability  $> 0.5 \rightarrow$  Class 1 (Yes); If  $< 0.5 \rightarrow$  Class 0 (No).

### Differences:

- Linear Regression → predicts continuous output (house price, score)
- Logistic Regression → predicts categories (spam/not spam, cancer/no cancer)
- Linear uses Ordinary Least Squares; Logistic uses Maximum Likelihood Estimation (MLE)

## Q: What is the Sigmoid Function?

The **Sigmoid (Logistic) Function** maps any real number to a value between 0 and 1.

Shape: 'S'-shaped curve.

- If input  $\rightarrow +\infty$ , output  $\rightarrow 1$
- If input  $\rightarrow -\infty$ , output  $\rightarrow 0$
- Output  $> 0.5 \rightarrow$  predict Class 1; Output  $< 0.5 \rightarrow$  predict Class 0

Example: Output = 0.75 means there is a 75% chance the patient has cancer.

### Q: What are the 3 types of Logistic Regression?

- ### 1. Binary Logistic Regression: Target has exactly 2 classes.

Example: Spam or Not Spam, Cancer or No Cancer.

- ## 2. Multinomial Logistic Regression: Target has 3 or more unordered categories.

Example: Type of wine (Red, White, Rosé).

- ### 3. Ordinal Logistic Regression: Target has 3 or more ordered categories.

Example: Restaurant rating (1, 2, 3, 4, 5 stars).

### Intermediate Questions

**Q: What is a Confusion Matrix? Draw and explain it.**

A **Confusion Matrix** is a table that shows the performance of a classification model by comparing actual vs predicted values.

For binary classification, it looks like:

[illegible]

■ Actual\Predicted ■ Predicted Yes ■ Predicted No ■

[illegible]

■ Actual Yes ■ TP (True Pos.) ■ FN (False Neg.)■

■ Actual No ■ FP (False Pos.) ■ TN (True Neg.) ■

[illegible]

- **TP (True Positive):** Predicted Yes, Actual Yes ✓
- **TN (True Negative):** Predicted No, Actual No ✓
- **FP (False Positive):** Predicted Yes, Actual No ✗ (Type I Error)
- **FN (False Negative):** Predicted No, Actual Yes ✗ (Type II Error)

**Q: Explain Accuracy, Precision, Recall, and F-Measure with formulas.**

**Accuracy:** Percentage of correct predictions out of all predictions.

Formula:  $(TP + TN) / (TP + TN + FP + FN)$

Example:  $(1+2)/(1+2+1+1) = 3/5 = 60\%$

**Precision:** Out of all predicted positives, how many were actually positive?

Formula:  $TP / (TP + FP)$

Example:  $1/(1+1) = 50\%$ . High precision = fewer false alarms.

**Recall (Sensitivity):** Out of all actual positives, how many did we correctly find?

Formula:  $TP / (TP + FN)$

Example:  $1/(1+1) = 50\%$ . High recall = fewer missed positives.

**F-Measure (F1 Score):** Harmonic mean of Precision and Recall. Used when both matter.

Formula:  $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$

Example:  $2 \times 0.5 \times 0.5 / (0.5 + 0.5) = 0.5 = 50\%$

## Advanced Questions

### Q: When should you prefer Recall over Precision?

**Recall is preferred** when missing a positive case is very costly (False Negatives are dangerous).

Example: **Cancer Detection** – Missing a cancer case (FN) is far worse than a false alarm (FP). So maximize Recall.

**Precision is preferred** when false alarms are costly.

Example: **Spam Detection** – Incorrectly marking a legitimate email as spam (FP) is worse. So maximize Precision.

When both matter equally, use **F1 Score**.

### Q: What dataset is used in Assignment 5 and what is the goal?

Dataset: **Social\_Network\_Ads.csv**

Goal: Use Logistic Regression to predict whether a user will purchase a product based on their Age and Estimated Salary.

Steps: Load data → Split into train/test → Apply LogisticRegression() → Predict → Build Confusion Matrix → Calculate TP, FP, TN, FN, Accuracy, Precision, Recall, F1 Score.

## ASSIGNMENT 6 – Naive Bayes Classifier

### Basic Questions

#### Q: What is the Naive Bayes Classifier?

**Naive Bayes** is a classification algorithm based on **Bayes' Theorem**. It assumes all features are independent of each other (the 'Naive' assumption).

It calculates the probability of each class given the features and predicts the class with the highest probability.

It is fast, simple, works well for text classification and small datasets.

#### Q: What is Bayes' Theorem? State it simply.

**Bayes' Theorem** calculates the probability of an event based on prior knowledge of related conditions.

Formula:  $P(\text{Class} | \text{Features}) = P(\text{Features} | \text{Class}) \times P(\text{Class}) / P(\text{Features})$

In simple words: **Posterior = Likelihood × Prior / Evidence**

- **Prior probability P(Class):** Probability of class before seeing features
- **Likelihood P(Features|Class):** Probability of seeing these features given this class
- **Posterior P(Class|Features):** Updated probability after seeing features

#### Q: Explain the Tennis Playing example used in Naive Bayes.

Dataset features: Outlook (Sunny/Overcast/Rain), Temperature, Humidity, Windy.

Target: Play Tennis? (Yes/No)

To predict for a new day:

1. Calculate  $P(\text{Yes})$  and  $P(\text{No})$  from training data
2. For each feature value, calculate  $P(\text{feature} | \text{Yes})$  and  $P(\text{feature} | \text{No})$
3. Multiply all probabilities together:  $P(\text{Yes} | \text{Today}) = P(\text{Yes}) \times P(\text{Outlook}|\text{Yes}) \times P(\text{Temp}|\text{Yes}) \times \dots$
4. Compare  $P(\text{Yes}|\text{Today})$  vs  $P(\text{No}|\text{Today})$
5. Predict the class with higher probability

If  $P(\text{No}|\text{Today}) > P(\text{Yes}|\text{Today}) \rightarrow \text{Predict: No Tennis}$

### Intermediate Questions

#### Q: What is the difference between Prior and Conditional Probability?

**Prior Probability:** Probability calculated without considering any other event.

Example:  $P(\text{Yes}) = 9/14 = 0.64$ ,  $P(\text{No}) = 5/14 = 0.36$  (from 14 tennis records)

No conditions involved.

**Conditional Probability:** Probability of an event given that another event has occurred.

Example:  $P(\text{Sunny} | \text{Yes}) = 2/9 = \text{probability of it being Sunny, given Tennis was played.}$

Used to calculate the likelihood in Bayes' theorem.

#### Q: Write the algorithm for Naive Bayes on the Iris Dataset.

Step 1: Import libraries (pandas, sklearn)

Step 2: Load Iris dataset

Step 3: Data Preprocessing – check null values, encode categorical variables if any

Step 4: Divide into X (features: sepal/petal measurements) and y (Species)

Step 5: Split into 80% train, 20% test using `train_test_split()`

Step 6: Create model: `from sklearn.naive_bayes import GaussianNB; gaussian = GaussianNB(); gaussian.fit(X_train, y_train)`

Step 7: Predict: `y_pred = gaussian.predict(X_test)`

Step 8: Evaluate: `accuracy_score()`, `precision_score()`, `recall_score()`, `confusion_matrix()`

### Q: Find the probability of car theft for: Red, SUV, Domestic (Car Theft Example).

Step 1: Count  $P(\text{Theft}=\text{Yes})$  and  $P(\text{Theft}=\text{No})$  from the dataset.

Step 2: Calculate conditional probabilities for each attribute:

- $P(\text{Red} \mid \text{Yes})$ ,  $P(\text{Red} \mid \text{No})$
- $P(\text{SUV} \mid \text{Yes})$ ,  $P(\text{SUV} \mid \text{No})$
- $P(\text{Domestic} \mid \text{Yes})$ ,  $P(\text{Domestic} \mid \text{No})$

Step 3:  $P(\text{Yes} \mid \text{Red}, \text{SUV}, \text{Domestic}) \propto P(\text{Yes}) \times P(\text{Red} \mid \text{Yes}) \times P(\text{SUV} \mid \text{Yes}) \times P(\text{Domestic} \mid \text{Yes})$

Step 4:  $P(\text{No} \mid \text{Red}, \text{SUV}, \text{Domestic}) \propto P(\text{No}) \times P(\text{Red} \mid \text{No}) \times P(\text{SUV} \mid \text{No}) \times P(\text{Domestic} \mid \text{No})$

Step 5: Compare the two values. The higher one is the predicted class.

## Advanced Questions

### Q: What is GaussianNB? When is it used?

**GaussianNB** (Gaussian Naive Bayes) assumes that the features follow a normal (Gaussian) distribution.

Used when: Features are continuous numeric values (like sepal length, petal width in Iris).

Other types:

- **MultinomialNB**: Used for discrete count data (e.g., word counts in text).
- **BernoulliNB**: Used for binary features (0/1 values).

## ASSIGNMENT 7 – Text Analytics (NLP)

### Basic Questions

#### Q: What is Text Analytics / NLP?

**Natural Language Processing (NLP)** is a field of AI that enables computers to understand, interpret, and generate human language.

Applications: Spam detection, sentiment analysis, language translation, chatbots, text classification.

**NLTK** (Natural Language Toolkit) is the most popular Python library for NLP.

#### Q: What is Tokenization? Explain its types.

**Tokenization** is the process of breaking text into smaller units called tokens (words or sentences). It is the first step in text processing.

**1. Sentence Tokenization:** Breaks a paragraph into individual sentences.

`from nltk.tokenize import sent_tokenize`

`sent_tokenize('Hello world. How are you?') → ['Hello world.', 'How are you?']`

**2. Word Tokenization:** Breaks text into individual words.

`word_tokenize('Hello world') → ['Hello', 'world']`

#### Q: What are Stop Words? Why are they removed?

**Stop words** are common words that appear very frequently but carry little meaning.

Examples: 'is', 'am', 'are', 'the', 'a', 'an', 'this', 'that', 'it', 'in', 'on'

They are **removed** because:

- They add noise without adding useful information
- They increase processing time unnecessarily
- They reduce accuracy of text classification

In NLTK: `from nltk.corpus import stopwords; stop_words = stopwords.words('english')`

### Intermediate Questions

#### Q: What is Stemming? What is Lemmatization? What is the difference?

**Stemming:** Reduces words to their root by removing suffixes/prefixes. It is fast but may give non-real words.

Example: 'connection', 'connected', 'connecting' → all become 'connect'

'studies' → 'studi' (not a real word, but stemming doesn't care)

**Lemmatization:** Reduces words to their base form (lemma) using dictionary lookup. Slower but more accurate.

Example: 'better' → 'good' (lemma), 'running' → 'run'

**Difference:** Stemming is faster but less accurate (may produce non-words). Lemmatization is slower but produces real words and considers the context (Part of Speech).

#### Q: What is POS Tagging?

**POS (Part of Speech) Tagging** identifies the grammatical role of each word in a sentence.

Tags: NOUN, VERB, ADJECTIVE, ADVERB, PRONOUN, PREPOSITION, etc.

Example: 'The cat sat on the mat'

→ The(DT), cat(NN), sat(VBD), on(IN), the(DT), mat(NN)

In NLTK: `nltk.pos_tag(tokens)`

POS tagging helps Lemmatization understand the correct meaning of words.

### Q: What is TF-IDF? Explain Term Frequency and Inverse Document Frequency.

**TF-IDF** is used to convert text into numbers (vectors) that represent the importance of each word in a document.

**TF (Term Frequency):** How often a word appears in a single document. Higher TF = word appears more in that document.

**IDF (Inverse Document Frequency):** Penalizes words that appear in many documents (common words). Words appearing in all documents get low IDF.

**TF-IDF = TF × IDF**

High TF-IDF → word is important in that document but rare in others = signature word.

In Python: `from sklearn.feature_extraction.text import TfidfVectorizer`

### Advanced Questions

#### Q: What is Bag of Words (BoW) Model?

**Bag of Words** creates a Document-Term Matrix by counting word occurrences in each document.

Example 3 documents:

- Doc1: 'I love dogs'
- Doc2: 'I hate dogs and knitting'
- Doc3: 'Knitting is my hobby'

→ Creates a matrix where each row is a document and each column is a word, with the value being the count.

BoW does not consider word order or meaning. TF-IDF is an improvement over BoW.

#### Q: What is the full text analytics pipeline for NLP classification?

1. **Load Data** (CSV/text file)
2. **Tokenization** – split into words/sentences
3. **Remove stop words**
4. **Stemming/Lemmatization** – reduce to root form
5. **POS Tagging** (optional)
6. **Feature Extraction** – convert text to numbers using BoW or TF-IDF
7. **Train/Test Split**
8. **Build Model** (e.g., Multinomial Naive Bayes)
9. **Evaluate** – accuracy, precision, recall, F1



## ASSIGNMENT 8 – Data Visualization I (Titanic + Histogram)

### Basic Questions

#### Q: What is Data Visualization? Why is it important?

**Data Visualization** is the process of representing data graphically (through charts, graphs, maps) to find trends, patterns, and insights that are hard to see in raw numbers.

##### Importance:

- Human brain processes visuals much faster than numbers
- Helps identify patterns, correlations, and outliers quickly
- Makes communication of data insights easier
- Especially useful for large datasets where manual inspection is impossible

#### Q: What are the main Python libraries for Data Visualization?

**1. Matplotlib:** Basic plotting library. Good for line charts, bar graphs, scatter plots, histograms. More customizable, works directly with arrays.

**2. Seaborn:** Built on top of Matplotlib. Better for statistical visualizations. Works with entire datasets. Has beautiful default themes. Less code needed.

**3. Plotly:** Interactive visualizations (hover, zoom, etc.)

For DSBD, mainly Matplotlib and Seaborn are used.

#### Q: What is the Titanic Dataset?

The **Titanic dataset** is a famous dataset available in Seaborn's inbuilt datasets.

It contains information about 891 passengers who boarded the Titanic ship.

Key columns: survived, pclass, sex, age, sibsp, parch, fare, embarked, class, who, adult\_male, deck, embark\_town, alive, alone.

Load using: `import seaborn as sns; titanic = sns.load_dataset('titanic')`

### Intermediate Questions

#### Q: What is a Histogram? When is it used? How to plot it?

A **Histogram** shows the distribution (frequency) of a continuous numerical variable.

The x-axis shows the value ranges (bins) and y-axis shows how many data points fall in each range.

**When used:** To understand the distribution of a single variable (is it normal? skewed? uniform?)

Using Matplotlib:

```
plt.hist(titanic['fare'], bins=30)
plt.xlabel('Fare'); plt.ylabel('Count'); plt.title('Fare Distribution')
plt.show()
```

Using Seaborn:

```
sns.histplot(data=titanic, x='fare', bins=30)
```

#### Q: Compare Matplotlib and Seaborn.

##### Matplotlib:

- More customizable and flexible
- Works directly with arrays and data frames
- Best for basic plots (line, bar, scatter, histogram)
- More code required

##### Seaborn:

- Built on Matplotlib, simpler syntax
- Works with entire datasets directly
- Better for statistical plots (box plots, violin plots, heatmaps)
- Beautiful default themes with `sns.set_style()`
- Less code required

## Advanced Questions

### Q: How do you use Seaborn to explore patterns in the Titanic dataset?

1. **Distribution of fares:** `sns.histplot(data=titanic, x='fare')`
2. **Survival count by gender:** `sns.countplot(data=titanic, x='sex', hue='survived')`
3. **Fare vs Class:** `sns.boxplot(data=titanic, x='class', y='fare')`
4. **Correlation heatmap:** `sns.heatmap(titanic.corr(), annot=True)`
5. **Set theme:** `sns.set_style('whitegrid')` for cleaner-looking plots

### Q: What are the different types of plots and when to use each?

- **Histogram:** Distribution of one numerical variable
- **Box Plot:** Distribution, quartiles, and outliers of numerical data
- **Scatter Plot:** Relationship between two numerical variables
- **Bar Chart:** Comparing values across categories
- **Line Plot:** Trends over time
- **Pie Chart:** Proportions/percentages of categories
- **Heatmap:** Correlation between multiple variables
- **Violin Plot:** Like box plot but shows full distribution shape

## ASSIGNMENT 9 – Data Visualization II (Box Plot)

### Basic Questions

#### Q: What is a Box Plot? What does it show?

A **Box Plot** (Box-and-Whisker Plot) is a graph that summarizes the distribution of data using 5 values:

- **Minimum:** Lowest value (excluding outliers) – left whisker end
- **Q1 (25th percentile):** Left edge of the box
- **Median (Q2):** Line inside the box
- **Q3 (75th percentile):** Right edge of the box
- **Maximum:** Highest value (excluding outliers) – right whisker end
- **Outliers:** Dots beyond the whiskers

The box contains the middle 50% of data ( $IQR = Q3 - Q1$ ).

#### Q: How to plot a box plot for Titanic's Age grouped by Gender and Survival?

```
sns.boxplot(data=titanic, x='sex', y='age', hue='survived')  
plt.title('Age Distribution by Gender and Survival')  
plt.show()
```

This shows 4 boxes: Male-Survived, Male-Died, Female-Survived, Female-Died, helping you visually compare age distributions across groups.

### Intermediate Questions

#### Q: Write observations from the Age vs Gender vs Survival box plot of Titanic.

Typical observations:

1. **Children were more likely to survive** – lower age boxes for 'survived=1'
2. **Women had higher survival rate** – 'women and children first' policy
3. **Median age of male survivors is lower** than male non-survivors
4. **Outliers visible** – some very old passengers in both groups
5. The spread (IQR) of female survivors is wider, indicating survival was across all age groups for women.

#### Q: What is the difference between a Histogram and a Box Plot?

##### Histogram:

- Shows the full frequency distribution (shape of data)
- X-axis = value ranges, Y-axis = count
- Good for seeing if data is normal, skewed, bimodal
- Does NOT show quartiles or outliers directly

##### Box Plot:

- Shows 5-number summary (min, Q1, median, Q3, max) + outliers
- Better for comparing distributions across multiple groups
- Clearly shows outliers as individual points
- Does NOT show the actual shape/frequency of distribution

## ASSIGNMENT 10 – Data Visualization III (Iris Dataset)

### Basic Questions

#### Q: What is the aim of Assignment 10?

Download the Iris dataset and perform the following:

1. List all features and their data types (numeric/nominal)
2. Create a histogram for each feature
3. Create a box plot for each feature
4. Compare distributions and identify outliers

#### Q: List the features of the Iris dataset and their types.

- **Id:** Integer – Nominal (just an identifier, no mathematical meaning)
- **SepalLengthCm:** Float – Continuous Numeric
- **SepalWidthCm:** Float – Continuous Numeric
- **PetalLengthCm:** Float – Continuous Numeric
- **PetalWidthCm:** Float – Continuous Numeric
- **Species:** String – Nominal Categorical (Iris-setosa, Iris-versicolor, Iris-virginica)

### Intermediate Questions

#### Q: How do you create a histogram for each feature in the Iris dataset?

Method 1 – Using Matplotlib:

```
iris.hist(figsize=(10,8))  
plt.show()
```

Method 2 – For each column individually:

```
for col in ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']:  
    plt.figure(); iris[col].plot(kind='hist'); plt.title(col); plt.show()
```

Method 3 – Using Seaborn:

```
sns.histplot(data=iris, x='SepalLengthCm', hue='Species')
```

#### Q: How do you create a box plot for each feature and detect outliers?

Method 1 – All features at once:

```
iris.drop('Id',axis=1).boxplot(figsize=(10,6))  
plt.show()
```

Method 2 – Seaborn box plot for each feature by species:

```
sns.boxplot(data=iris, x='Species', y='SepalLengthCm')
```

Outliers appear as individual dots (circles) beyond the whiskers of the box plot.

### Advanced Questions

#### Q: Write observations from histograms and box plots of the Iris dataset.

**From Histograms:**

- Sepal Width is approximately normally distributed
- Petal Length and Petal Width have two peaks (bimodal) – indicating two groups (setosa vs others)

- Iris-setosa is clearly separated from other species in petal measurements

**From Box Plots:**

- SepalWidthCm has visible outliers (dots beyond whiskers)
- Petal features show clear separation between Iris-setosa and other two species
- Iris-virginica has the largest petal length and width
- Iris-setosa has the smallest petals but widest sepals

**Q: How do you compare distributions across the 3 Iris species?**

**1. Using describe() per species:**

```
iris.groupby('Species').describe()
```

**2. Using Seaborn violin plot:** Shows full distribution shape + box plot in one

```
sns.violinplot(data=iris, x='Species', y='PetalLengthCm')
```

**3. Using pair plot:** Shows relationships between ALL pairs of features, colored by species

```
sns.pairplot(iris, hue='Species')
```

This is the most powerful visualization for multi-feature comparison and cluster detection.

## QUICK REVISION – Most Important One-Liners

**Dataset:** Collection of records arranged in rows (instances) and columns (features)

**DataFrame:** 2D table in Pandas – like an Excel sheet

**isnull():** Checks for missing/NaN values

**fillna():** Fills missing values with a given value or statistic

**dropna():** Removes rows or columns with missing values

**Outlier:** A data point that is very far from the rest of the data

**IQR:**  $Q3 - Q1$ ; used to define upper/lower bounds for outlier detection

**Normalization:** Scaling data to 0-1 range using Min-Max scaling

**Label Encoding:** Converts categories to numbers (0, 1, 2...) – creates false order

**One-Hot Encoding:** Creates binary columns for each category – no false order

**Mean:** Sum / Count – affected by outliers

**Median:** Middle value – not affected by outliers

**Mode:** Most frequent value

**Standard Deviation:** Spread of data around mean; sqrt of variance

**Skewness:** Asymmetry in data distribution

**Linear Regression:** Predicts continuous output ( $Y = mX + b$ )

**Logistic Regression:** Predicts categories (uses sigmoid function)

**Confusion Matrix:** Table comparing actual vs predicted class labels

**Accuracy:**  $(TP+TN)/(TP+TN+FP+FN)$

**Precision:**  $TP/(TP+FP)$  – out of predicted positives, how many are correct

**Recall:**  $TP/(TP+FN)$  – out of actual positives, how many were found

**F1 Score:**  $2 \times P \times R / (P + R)$  – balance between precision and recall

**Naive Bayes:** Probabilistic classifier based on Bayes Theorem; assumes feature independence

**Tokenization:** Breaking text into words or sentences

**Stop Words:** Common words (the, is, a) removed as they add no meaning

**Stemming:** Reducing words to root form (running → run) – fast but may give non-words

**Lemmatization:** Reducing words to dictionary base form (better → good) – accurate

**TF-IDF:** Measures word importance in a document relative to all documents

**Histogram:** Bar chart showing frequency distribution of one variable

**Box Plot:** Shows min, Q1, median, Q3, max, and outliers of a variable

---

**All the Best for Your Practical Exam! You've Got This!**